fMRI × DINOv2 — Code Review

Code Review — our fMRI changes on top of DINOv2

Every change we made to the official DINOv2 (`facebookresearch/dinov2`), traced in the **order the data flows through the pipeline**. Tags (**verified against** `upstream/main`): 🟢 **new file** (entirely ours) · 🟠 **official file we modified** · 🔵 **config**.

Each block is shown as: the **actual code** · its **exact location** (path + lines) · a **complete explanation** of what it does (nothing skipped) · **where & why it's used**.

> **Known limitation (see `tasks/v3/FINDINGS.md`):** the masking-only design of Phase 5 (all DINO views = the same full volume) leaves DINO/iBOT with no augmentation gap, so the SSL loss does not descend. Not a bug — the fix is to restore a real view gap (temporal-window crops / 3D augmentations).

Phase 1 — Corpus construction

`fmri_const` constants block — corpus paths, TR tables, harmonization targets

📁 `dinov2/data/fmri_const.py` · lines **10–51** · 🟢 new file

```
LAB_ROOT = "/sci/labs/arieljaffe/dan.abergel1"
TARGET_TR = 0.72                      # common TR after harmonization (HCP native)
TARGET_SHAPE = (45, 54, 45)           # fixed spatial resolution (X, Y, Z)
DEFAULT_T_FIXED = 270                 # window = 270 frames @ 0.72s = 194.4s
DEFAULT_MANIFEST = "corpus_manifest.csv"   # under LAB_ROOT
DEFAULT_SPLIT = "subject_split.json"       # under LAB_ROOT
CORPUS_DATASETS = ("HCP", "ABIDE", "OASIS", "AOMIC", "ADNI")
HOLDOUT_DATASETS = ("ADNI", "ABIDE", "OASIS", "HCP")   # their TEST subjects leave pretraining
PRETRAIN_SPLITS = ("train",)   # which splits enter pretraining (test is held out)
DROP_SHORT = True              # drop scans too short to fill a T_FIXED window
GLOBAL_CROPS_NUMBER = 2        # DINO invariant (2 global views), not a config knob
ABIDE_SITE_TR = { "Caltech": 2.0, "CMU": 2.0, ... "Yale": 2.0 }   # per-site native TR
OASIS_DEFAULT_TR = 2.2
AOMIC_TR = {"piop1": 0.75, "piop2": 2.0}
HCP_TR = 0.72
ADNI_TR = 3.0
```

What it does:

- `LAB_ROOT` — cluster root holding every `<DATASET>_data/downsampled/` folder + the manifest and split; the default `root` when none is passed to `MixedFMRIDataset` .
- `TARGET_TR = 0.72` — the common repetition time (s/volume) every dataset is harmonized to (HCP native); used for the native-window length and the offline `upsampled_T` formula.
- `TARGET_SHAPE = (45, 54, 45)` — the fixed spatial grid; `_finalize` trilinearly resizes any scan not already at it.
- `DEFAULT_T_FIXED = 270` — harmonized window length in frames (270 × 0.72 = 194.4 s); tuned 1 frame below ABIDE's min upsampled length (271) so all of ABIDE survives, dropping only 2 short OASIS scans.
- `DEFAULT_MANIFEST` / `DEFAULT_SPLIT` — manifest/split filenames, resolved under `LAB_ROOT` .
- `CORPUS_DATASETS` — the five pretraining sources; default filter for discovery + manifest.
- `HOLDOUT_DATASETS` — the four cohorts whose TEST subjects are excluded from pretraining (AOMIC absent → kept whole, no downstream probe).
- `PRETRAIN_SPLITS = ("train",)` — splits allowed into pretraining; any "test" subject of a holdout dataset is dropped.
- `DROP_SHORT = True` — toggles the length filter (drop scans too short for a window).
- `GLOBAL_CROPS_NUMBER = 2` — DINO teacher-forward invariant; consumed by `FullVolumeViews3D` , deliberately not a config knob.

- `ABIDE_SITE_TR` / `OASIS_DEFAULT_TR` / `AOMIC_TR` / `HCP_TR` / `ADNI_TR` — native TR tables consumed by the `DATASET_SOURCES` `tr` lambdas.

Where & why it's used:

- **Imported by** `fmri_offline.py` and re-exported through `fmri_data.py:67–70`.
- **Why:** single source of truth for the corpus. Keeping filters + TR tables here (not as dataset args) is what lets `MixedFMRIDataset.__init__` take almost no arguments — changing corpus behavior means editing this one file.

`DATASET_SOURCES` — declarative per-dataset scan discovery table

📁 `dinov2/data/fmri_offline.py` · lines **32–53** · 🟢 new file

```
DATASET_SOURCES = {
    "HCP":   dict(glob="HCP_data/downsampled/subject_*/rfMRI_REST1_LR_downsampled.pt",
                  subject=lambda p: p.parent.name,          tr=lambda p: HCP_TR),
    "ABIDE": dict(glob="ABIDE_data/downsampled/**/*.pt",
                  subject=lambda p: p.stem,                 tr=lambda p: ABIDE_SITE_TR.get(p.name.split("_")[0])),
    "OASIS": dict(glob="OASIS3_data/downsampled/*/rest_*.pt",
                  subject=lambda p: p.parent.name,          tr=lambda p: OASIS_DEFAULT_TR),
    "AOMIC": dict(glob="AOMIC_data/downsampled/*/sub-*/restingstate_downsampled.pt",
                  subject=lambda p: p.parent.name,          tr=lambda p: AOMIC_TR["piop1" if "piop1" in str(p).lower()
        else "piop2"]),
    "ADNI":  dict(glob="ADNI_data/downsampled/*/I*.pt",
                  subject=lambda p: p.parent.name,          tr=lambda p: ADNI_TR),
}
```

What it does:

- One declarative entry per cohort (three fields each) — adding a cohort is one line, no new `if/elif`.
- `glob` — path pattern (relative to `lab_root`) matching every scan `.pt` of the cohort.
- `subject` — lambda deriving the subject id from a `Path` (parent folder name, or filename stem for ABIDE). All scans of one subject share this id so they never straddle the train/test split.
- `tr` — lambda returning the native TR: constant for HCP/OASIS/ADNI, per-protocol for AOMIC (piop1/piop2 from the path), per-site for ABIDE (returns `None` for an unmapped site).

Where & why it's used:

- **Iterated by** `build_corpus_entries` (loops `sorted(Path(lab_root).glob(src["glob"]))`, calling `src["tr"](p)` / `src["subject"](p)`).
- **Why:** the registry turning an on-disk cohort layout into a uniform `{dataset, path, subject_id, tr}` scan list, keeping discovery data-driven.

`build_corpus_entries` — discover every scan on disk into a flat list

📁 `dinov2/data/fmri_offline.py` · lines **56–86** · 🟢 new file

```
def build_corpus_entries(lab_root=LAB_ROOT, datasets=CORPUS_DATASETS):
    entries = []
    for name in datasets:
        src = DATASET_SOURCES.get(name)
        if src is None:                                  # unknown dataset name -> skip
            continue
        for p in sorted(Path(lab_root).glob(src["glob"])): # sorted() -> deterministic corpus
            tr = src["tr"](p)
            if tr is None:                               # e.g. an unmapped ABIDE site
                logger.warning(f"{name}: no native TR for {p.name}; skipping")
                continue
            entries.append({"dataset": name, "path": str(p),
                            "subject_id": src["subject"](p), "tr": float(tr)})
    return entries
```

What it does:

- Loops the requested `datasets` (default all five); an unknown name is skipped.
- Globs each cohort's pattern under `lab_root`, iterating matches in `sorted()` order → deterministic/reproducible corpus.
- Resolves each scan's native TR; if `None` (unmapped ABIDE site) logs a warning and skips.
- Appends one dict per scan `{dataset, path (str), subject_id, tr (float)}`; returns the flat list.

Where & why it's used:

- **Called by** `write_corpus_manifest` and `compute_t_fixed_max`.
- **Why:** the single offline discovery pass; both the manifest writer and the T_fixed calculator build on the same deterministic scan list so they agree on the corpus.

`write_corpus_manifest` — scan native lengths once → `corpus_manifest.csv`

📁 `dinov2/data/fmri_offline.py` · lines **89–112** · 🟢 new file

```
def write_corpus_manifest(out_path, lab_root=LAB_ROOT, datasets=CORPUS_DATASETS):
    entries = build_corpus_entries(lab_root, datasets)
    out_path = Path(out_path); out_path.parent.mkdir(parents=True, exist_ok=True)
    with open(out_path, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["dataset", "path", "subject_id", "tr", "T_native", "upsampled_T"])
        for n, e in enumerate(entries, 1):
            T = int(_load_mmap(e["path"]).shape[0])
            w.writerow([e["dataset"], e["path"], e["subject_id"], e["tr"], T,
                        round(T * e["tr"] / TARGET_TR)])
            if n % 200 == 0:
                logger.info(f"  corpus manifest: {n}/{len(entries)}")
    logger.info(f"corpus manifest written: {len(entries)} scans -> {out_path}")
    return out_path
```

What it does:

- Builds the scan list, creates the output dir if missing, opens the CSV.
- Writes a header `dataset, path, subject_id, tr, T_native, upsampled_T`.
- For each scan: mmaps it and reads only `shape[0]` = native frame count `T` (no full load).
- Computes `upsampled_T = round(T * tr / TARGET_TR)` (frame count after harmonization to 0.72 s), writes one row per scan; logs progress every 200; returns the path.

Where & why it's used:

- **Offline entry point** (no in-repo caller) — run once/when the corpus changes; its CSV is read at training time by `entries_from_manifest`.
- **Why:** pre-computes each scan's native and upsampled length so training-time filtering (drop-short, holdout) runs without opening any `.pt` file.

`compute_t_fixed_max` — largest window that fits every scan (picks T=270)

📁 `dinov2/data/fmri_offline.py` · lines **115–128** · 🟢 new file

```
def compute_t_fixed_max(lab_root=LAB_ROOT, datasets=CORPUS_DATASETS, margin=0):
    entries = build_corpus_entries(lab_root, datasets)
    per, per_all, g_min, argmin = {}, {}, None, None
    for e in entries:
        up = round(_load_mmap(e["path"]).shape[0] * e["tr"] / TARGET_TR)
        d = e["dataset"]
        per[d] = min(per.get(d, up), up)
        per_all.setdefault(d, []).append(up)
        if g_min is None or up < g_min:
            g_min, argmin = up, e
    return max(1, g_min - margin), per, argmin, per_all
```

What it does:

- For each scan computes its upsampled length `up = round(T_native * tr / TARGET_TR)` (mmap, reads only the frame count).
- Tracks per-dataset minimum (`per`), all per-dataset lengths (`per_all`), the global minimum (`g_min`) and the shortest scan (`argmin`).
- Returns `(max(1, g_min - margin), per, argmin, per_all)` — the largest T_fixed fitting every scan with no padding (minus an optional margin) + diagnostics.

Where & why it's used:

- **Offline analysis helper** (no in-repo caller) — run manually to justify `DEFAULT_T_FIXED = 270` .
- **Why:** determines the window-length knee so 270 keeps all of ABIDE while dropping only the two short OASIS outliers.

`_index_by_dataset` — group scan indices per cohort for the sampler

📁 `dinov2/data/fmri_data.py` · lines **80–100** · 🟢 new file

```
def _index_by_dataset(entries):
    by = {}
    for i, e in enumerate(entries):
        by.setdefault(e["dataset"], []).append(i)
    return by
```

What it does: single pass over the flat `entries` , bucketing each entry's position `i` by `dataset` → `{dataset: [indices]}` (e.g. `{"HCP": [0, 2], "ABIDE": [1]}`).

Where & why it's used:

- **Called by** `MixedFMRIDataset._discover` , stored as `self.dataset_indices` .
- **Why:** `ProportionalInfiniteSampler` needs to know which flat indices belong to which cohort to draw its per-dataset quota; this derives that mapping once.

`_load_split_map` — invert `subject_split.json` into an O(1) per-subject lookup

📁 `dinov2/data/fmri_data.py` · lines **103–122** · 🟢 new file

```
def _load_split_map(split_file):
    d = json.loads(Path(split_file).read_text())
    return {ds: {s: name for name, subs in splits.items() for s in subs}
            for ds, splits in d.get("datasets", {}).items()}
```

What it does:

- Reads/parses `subject_split.json` shaped `{"datasets": {<ds>: {"train": [...], "test": [...]}}}` .

- Inverts each per-dataset `{split: [subjects]}` into `{subject: split}` → `{dataset: {subject_id: "train"|"test"}}` for O(1) checks.
- `d.get("datasets", {})` → a file without a `datasets` key yields an empty map rather than erroring.

Where & why it's used:

- **Called by** `MixedFMRIDataset._discover` ; the map is passed to `entries_from_manifest` .
- **Why:** enables subject-level holdout. Splitting by SUBJECT (not scan) prevents leakage — all scans of a held-out subject stay out together, checked in O(1).

`entries_from_manifest` — read the manifest, apply length + holdout filters

📁 `dinov2/data/fmri_data.py` · lines **125–172** · 🟢 new file

```
def entries_from_manifest(manifest_path, datasets=CORPUS_DATASETS, min_upsampled_t=0, split_map=None):
    entries, n_short, n_holdout = [], 0, 0
    with open(manifest_path) as f:
        for row in csv.DictReader(f):
            ds = row["dataset"]
            if ds not in datasets:                              continue
            if int(row["upsampled_T"]) < min_upsampled_t:                # filter 1: too short
                n_short += 1; continue
            if split_map and ds in HOLDOUT_DATASETS:                     # filter 2: holdout
                sp = split_map.get(ds, {}).get(row["subject_id"])
                if sp is not None and sp not in PRETRAIN_SPLITS:
                    n_holdout += 1; continue
            entries.append({"dataset": ds, "path": row["path"],
                            "subject_id": row["subject_id"], "tr": float(row["tr"])})
    ...
    return entries
```

What it does:

- Reads `corpus_manifest.csv` row by row (`csv.DictReader`); skips rows whose `dataset` isn't requested.
- **Filter 1 (LENGTH):** drops scans with `upsampled_T < min_upsampled_t` (270 at training) — too short to fill a window; counts `n_short` .
- **Filter 2 (HOLDOUT):** only for `HOLDOUT_DATASETS` and only with a `split_map` — looks up the subject's split; if defined and not in `PRETRAIN_SPLITS` (i.e. "test"), drops it; counts `n_holdout` . A subject absent from the map (`sp is None`) is kept.
- Survivors become `{dataset, path, subject_id, tr (float)}` ; both filters run without opening any scan (length pre-computed offline). Logs how many each filter dropped.

Where & why it's used:

- **Called by** `MixedFMRIDataset._discover` (`min_upsampled_t = self.t_fixed if DROP_SHORT else 0`).
- **Why:** the normal load path — turns the offline manifest into the exact filtered scan list the model sees, enforcing both the window-length requirement and the no-leakage holdout.

`MixedFMRIDataset.__init__` — minimal dataset constructor

📁 `dinov2/data/fmri_data.py` · lines **310–320** · 🟢 new file

```
def __init__(self, root=None, *, t_fixed=DEFAULT_T_FIXED, transform=None, **_ignored):
    self.t_fixed = int(t_fixed)                          # window length in frames (270)
    self.transform = transform                           # image augmentation, or None
    lab_root = root or LAB_ROOT
    self.entries, self.dataset_indices = self._discover(lab_root)
    if not self.entries:
        raise FileNotFoundError(f"No scans under {lab_root} for {CORPUS_DATASETS}")
```

What it does:

- Takes only what varies: `t_fixed` (from the config string) and `transform` (from `do_train`); `root` defaults to `LAB_ROOT`.
- `**_ignored` swallows any other kwargs — notably `target_transform`, since SSL-only means no label to transform.
- Stores `t_fixed` (int) + `transform`, runs `_discover` → `self.entries` (scan list) and `self.dataset_indices` (the sampler's contract); raises if no scans survive.

Where & why it's used:

- **Instantiated by** `make_dataset` (loaders.py) after `_parse_dataset_str` resolves `"Mixed"`.
- **Why:** presents the five cohorts as one dataset with the same `(root/transform)` API as ImageNet so it drops into `do_train` unchanged, pushing corpus policy into constants. Exposing `dataset_indices` is what enables the proportional sampler.

`MixedFMRIDataset._discover` — split → manifest → per-dataset index (one-time setup)

📁 `dinov2/data/fmri_data.py` · lines **322–383** · 🟢 new file

```
def _discover(self, lab_root):
    sf = Path(lab_root) / DEFAULT_SPLIT                          # Step 1: holdout map
    split_map = _load_split_map(sf) if sf.exists() else None
    man = Path(lab_root) / DEFAULT_MANIFEST                      # Step 2: read manifest (required)
    if not man.exists():
        raise FileNotFoundError(f"No corpus manifest at {man}. Build it offline first ...")
    entries = entries_from_manifest(
        man, min_upsampled_t=(self.t_fixed if DROP_SHORT else 0), split_map=split_map)
    overfit_n = int(os.environ.get("FMRI_OVERFIT_N", "0"))      # DEBUG overfit hook
    if overfit_n > 0:
        entries = [e for e in entries if e["dataset"] == "HCP"][:overfit_n]
        if not entries: raise RuntimeError("FMRI_OVERFIT_N set but no HCP scan found ...")
        logger.warning(f"FMRI_OVERFIT_N={overfit_n}: corpus truncated to {len(entries)} HCP scan(s) ...")
        for i, e in enumerate(entries):
            logger.warning(f"  overfit scan[{i}]  dataset={e['dataset']:6s} subject={e['subject_id']} ...")
    idx = _index_by_dataset(entries)                             # Step 3: index by dataset
    logger.info(f"MixedFMRIDataset: {len(entries)} scans  T_fixed={self.t_fixed}  ...")
    return entries, idx
```

What it does:

- **Step 1** — resolves `subject_split.json`; loads it (`_load_split_map`) if present, else `None` (keep every subject, no holdout).
- **Step 2** — resolves `corpus_manifest.csv`; raises `FileNotFoundError` with build instructions if absent (the manifest is REQUIRED — never globs the disk). Calls `entries_from_manifest` with `min_upsampled_t = self.t_fixed` when `DROP_SHORT`, plus the split map.
- **Overfit hook** — if env `FMRI_OVERFIT_N=k > 0`, truncates to the first `k` HCP scans for a memorization sanity check (raises if no HCP), logging each retained scan's dataset/subject/tr/path. Debug-only.
- **Step 3** — indexes survivors by dataset (`_index_by_dataset`), logs counts, returns `(entries, idx)`.

Where & why it's used:

- **Called by** `MixedFMRIDataset.__init__`.
- **Why:** the complete one-time corpus assembly (holdout + length filters + sampler index) before any sample is read, governed entirely by `fmri_const` constants.

`_parse_dataset_str` `"Mixed"` branch + `t_fixed` key — resolve config string to the dataset

📁 `dinov2/data/loaders.py` · lines **82–85** and **104–113** · 🟠 modified official file

```
    for token in tokens[1:]:
        key, value = token.split("=")
        # FMRI: "t_fixed" lets "Mixed:t_fixed=270" reach MixedFMRIDataset.
        assert key in ("root", "extra", "split", "mode", "wildcard", "t_fixed")
        kwargs[key] = value
...
    elif name == "Mixed":            # FMRI: resolve the multi-source corpus
        class_ = MixedFMRIDataset
```

What it does:

- Adds `"t_fixed"` to the allowed dataset-string keys, so `"Mixed:t_fixed=270"` passes the assertion and forwards `t_fixed` to the dataset (`__init__` casts to int).
- Adds an `elif name == "Mixed":` branch resolving the dataset name `"Mixed"` → `MixedFMRIDataset`.

Where & why it's used:

- **Called by** `make_dataset`; `cfg.train.dataset_path: "Mixed"` in the fMRI YAML flows here.
- **Why:** makes the multi-source corpus selectable from the config like any upstream dataset, keeping `make_dataset` / `do_train` fMRI-agnostic.

`ProportionalInfiniteSampler` — per-batch dataset-quota infinite sampler

📁 `dinov2/data/samplers.py` · lines **245–339** · 🟠 modified official file (new class)

```python
class ProportionalInfiniteSampler(Sampler):
    DEFAULT_QUOTA = {"HCP": 4, "ABIDE": 4, "OASIS": 4, "ADNI": 3, "AOMIC": 1}

    def __init__(self, *, dataset_indices, quota=None, seed=0, advance=0, rank=None):
        self._indices = {k: list(v) for k, v in dataset_indices.items() if v}
        quota = quota or self.DEFAULT_QUOTA
        self._quota = {k: int(q) for k, q in quota.items() if k in self._indices and q > 0}
        if not self._quota:
            raise ValueError(f"quota {list(quota)} matches none of datasets {list(self._indices)}")
        self._batch_size = sum(self._quota.values())
        self._seed, self._advance = seed, advance
        self._rank = distributed.get_global_rank() if rank is None else rank
        self._offset = {name: i for i, name in enumerate(sorted(self._quota))}

    @property
    def batch_size(self): return self._batch_size

    def _pool(self, name, cycle):        # rank- and cycle-dependent shuffle of one dataset
        g = torch.Generator().manual_seed(
            self._seed + 1_000_003 * (self._rank + 1) + 7_919 * self._offset[name] + cycle)
        idxs = self._indices[name]
        return [idxs[i] for i in torch.randperm(len(idxs), generator=g).tolist()]

    def _iterator(self):
        pools, ptr, cyc = {}, {}, {}
        for name in self._quota:
            pools[name], ptr[name], cyc[name] = self._pool(name, 0), 0, 0
        batch_idx = 0
        while True:
            batch = []
            for name, q in self._quota.items():
                pool, p = pools[name], ptr[name]
                for _ in range(q):
                    if p >= len(pool):                # exhausted -> reshuffle + cycle
                        cyc[name] += 1; pool = self._pool(name, cyc[name]); pools[name] = pool; p = 0
                    batch.append(pool[p]); p += 1
                ptr[name] = p
            g = torch.Generator().manual_seed(self._seed + 1_000_003 * (self._rank + 1) + 31 * batch_idx)
            order = torch.randperm(len(batch), generator=g).tolist(); batch_idx += 1
            for i in order: yield batch[i]

    def __iter__(self):
        yield from itertools.islice(self._iterator(), self._advance, None)
```

What it does:

- `DEFAULT_QUOTA` — per-batch counts HCP4/ABIDE4/OASIS4/ADNI3/AOMIC1 (sum 16) when no quota is passed.
- `__init__` — copies `dataset_indices` dropping empty cohorts; keeps only present datasets with count > 0 (else `ValueError`); computes `_batch_size = sum(quota)`; resolves the DDP `rank` (each rank runs its OWN stream, no `[start::step]` striding); builds `_offset` = a stable per-dataset integer keyed on `sorted` names for insertion-order-independent seeding.
- `batch_size` property — exposes `sum(quota)`; the loader validates `batch_size` divides it.
- `_pool(name, cycle)` — a rank- and cycle-dependent random permutation of one dataset's indices, seeded by `seed + 1_000_003·(rank+1) + 7_919·offset[name] + cycle` (different ranks/cycles → different deterministic orders).
- `_iterator()` — inits a shuffled pool + read pointer + cycle counter per dataset; loops forever building one quota block at a time (draws `q` from each dataset, reshuffling with an incremented cycle when a pool empties = sampling without replacement within a cycle); each block is then shuffled by a batch-index-seeded permutation (datasets interleaved, not grouped) and yielded index by index.
- `__iter__()` — wraps `_iterator()` with `islice(..., advance, None)` to skip `advance` indices for resumption.

Where & why it's used:

- **Constructed in** `_make_sampler` under `SamplerType.PROPORTIONAL`, selected in `train.py` when `cfg.train.proportional_sampler` is set and the dataset exposes `dataset_indices`.

- **Why:** the infinite, iteration-based sampler DINOv2's loop needs, giving every batch a fixed cohort composition so the five wildly-different-sized datasets each contribute a controlled share.

`SamplerType.PROPORTIONAL` branch + block-size validation — sampler wiring in the loader

📁 `dinov2/data/loaders.py` · lines **57, 161, 165–186, 255, 284–306** · 🟠 modified official file

```
    PROPORTIONAL = 5          # FMRI: per-batch dataset quota over MixedFMRIDataset
...
    if type == SamplerType.PROPORTIONAL:
        if not hasattr(dataset, "dataset_indices"):
            raise ValueError("SamplerType.PROPORTIONAL requires a dataset exposing `dataset_indices` ...")
        return ProportionalInfiniteSampler(
            dataset_indices=dataset.dataset_indices, quota=proportional_quota, seed=seed, advance=advance)
...
    if sampler_type == SamplerType.PROPORTIONAL:        # block-size validation in make_data_loader
        if sampler.batch_size % batch_size != 0:
            raise ValueError(f"sum(quota) ({sampler.batch_size}) must be a multiple of batch_size ({batch_size}) ...")
        n_micro = sampler.batch_size // batch_size
        logger.info(f"proportional sampler: block={sampler.batch_size}, batch_size={batch_size} -> ... {n_micro} micro-
        batches ...")
```

What it does:

- Adds `PROPORTIONAL = 5` to the `SamplerType` enum.
- `_make_sampler` gains a `proportional_quota` param + a branch: asserts the dataset has `dataset_indices` (else `ValueError`), logs, constructs `ProportionalInfiniteSampler`.
- `make_data_loader` gains a `proportional_quota` param, forwarded to `_make_sampler`.
- Block-size validation: requires `sampler.batch_size` (= `sum(quota)`) to be a multiple of the loader `batch_size` (else `ValueError`); logs `n_micro = sum(quota)//batch_size` and advises `grad_accum_steps = n_micro` so the quota composition realizes over that many micro-batches.

Where & why it's used:

- **Called from** `train.py`, which sets `sampler_type = PROPORTIONAL` + passes `proportional_quota` when `cfg.train.proportional_sampler` is on (else falls back to `SHARDED_INFINITE`).
- **Why:** lets the loop opt into per-batch quotas while guaranteeing the micro-batch size aligns with the quota block so the intended composition materializes.

Phase 2 — Loading one sample

`MixedFMRIDataset.__getitem__` and `._load` — one scan → one model-ready window

📁 `dinov2/data/fmri_data.py` · lines **385–411** · 🟢 new file

```
    def _load(self, idx):
        e = self.entries[idx]
        scan = _load_mmap(e["path"])                        # 1. lazy (T, X, Y, Z)
        start, win = _native_window(scan.shape[0], e["tr"], self.t_fixed)  # 2. window
        return _finalize(scan[start:start + win].clone(), self.t_fixed)    # 3. crop + prep

    def __getitem__(self, idx):
        image = self._load(idx)
        if self.transform is not None:
            image = self.transform(image)                   # augmentation (masking, Phase 5)
        return image, ()                                    # () instead of labels because SSL
```

What it does:

- `_load(idx)` : `e = self.entries[idx]` fetches the record; `_load_mmap` memory-maps the tensor (nothing in RAM yet); `_native_window` chooses a native-frame window (random start, native length ≈ 194.4 s); `scan[start:start+win]` slices ON THE MMAP first, then `.clone()` materializes only that slice (keeps a ~500 MB scan off RAM); `_finalize` produces `(T_fixed, 1, 45, 54, 45)` .
- `__getitem__(idx)` : `_load(idx)` ; apply `self.transform` (multi-view + masking) only if wired; return `(image, ())` — the SSL-only empty target the DINO collator tolerates (it reads `s[0]` only).

Where & why it's used:

- **Called by** PyTorch's `DataLoader` for every sampled index (from `ProportionalInfiniteSampler`).
- **Why:** the single per-sample loading path — converts any scan (arbitrary TR/length/resolution) into the fixed `(T_fixed, 1, 45, 54, 45)` tensor a mixed batch can stack, returning the SSL-safe empty target.

`_load_mmap` — lazy memory-map of a scan

📁 `dinov2/data/fmri_data.py` · lines **196–204** · 🟢 new file

```
def _load_mmap(path):
    return torch.load(path, map_location="cpu", weights_only=True, mmap=True)
```

What it does:

- `torch.load(path, ...)` : opens the `.pt` scan tensor.
- `map_location="cpu"` : keeps it on CPU (never touches the GPU at load).
- `weights_only=True` : safe deserialization (only tensor data, no arbitrary pickled objects).
- `mmap=True` : memory-maps the file — bytes are paged in only when actually indexed, so opening a large scan costs almost nothing until it is sliced.

Where & why it's used:

- **Called by** `MixedFMRIDataset._load` (step 1).
- **Why:** lazy mmap is what lets `_load` crop the window BEFORE `.clone()` , so only the ~270-frame window is materialized rather than a multi-hundred-MB scan.

`_native_window` — pick the temporal window (random crop)

📁 `dinov2/data/fmri_data.py` · lines **207–235** · 🟢 new file

```
def _native_window(T, tr_native, t_fixed):
    win = max(1, round(t_fixed * TARGET_TR / tr_native))
    if T >= win:
        if os.environ.get("FMRI_FIXED_WINDOW"):              # debug: pin start to 0
            return 0, win
        return int(np.random.randint(0, T - win + 1)), win   # random start
    return 0, T                                              # scan shorter than window → whole
```

What it does:

- `win = max(1, round(t_fixed * TARGET_TR / tr_native))` : converts the target duration (270 × 0.72 = 194.4 s) into NATIVE frames via the scan's TR (270 for HCP @0.72s, ~65 for ADNI @3.0s); `max(1, …)` guarantees ≥1 frame.
- `if T >= win` : is the scan long enough?
  - `FMRI_FIXED_WINDOW` : debug hook → pin start to 0 (identical input every load, for overfit tests — no temporal augmentation).
  - `np.random.randint(0, T - win + 1)` : else a RANDOM start in `[0, T−win]` → temporal augmentation.
- `return 0, T` : scan shorter than the window → take it whole (`_temporal_resample` stretches to 270).

Where & why it's used:

- **Called by** `MixedFMRIDataset._load` (every sample load).

- **Why:** decides which temporal slice to load; gives every dataset the same 194.4 s real-time coverage despite different TRs, plus temporal augmentation via the random start.

Phase 3 — TR harmonization

`_finalize` (incl. `F.interpolate` spatial resize) — raw window → model-ready tensor

📁 `dinov2/data/fmri_data.py` · lines **270–293** · 🟢 new file

```
def _finalize(clip, t_fixed):
    clip = clip.float()
    if clip.ndim == 4:                              # (n,X,Y,Z) -> add channel
        clip = clip.unsqueeze(1)
    if tuple(clip.shape[-3:]) != tuple(TARGET_SHAPE):
        clip = F.interpolate(clip, size=tuple(TARGET_SHAPE), mode="trilinear", align_corners=False)
    return _zscore_per_frame(_temporal_resample(clip, t_fixed))
```

What it does:

- `clip.float()` : promotes the cropped window to float (needed for interpolation + z-score).
- `if clip.ndim == 4: clip = clip.unsqueeze(1)` : scans stored `(n, X, Y, Z)` gain a channel → `(n, 1, X, Y, Z)` ; already-5D scans unchanged — normalizes the layout for `F.interpolate` (dim 0 = batch, dim 1 = channel).
- `if tuple(clip.shape[-3:]) != TARGET_SHAPE` : resize only when not already `(45, 54, 45)` (a no-op otherwise).
  - `F.interpolate(..., size=(45,54,45), mode="trilinear", align_corners=False)` : trilinear 3D resize per (frame, channel) — harmonizes SPATIAL resolution across cohorts.
- `return _zscore_per_frame(_temporal_resample(clip, t_fixed))` : chains temporal resample (Phase 3.2) then per-frame z-score (Phase 4.1) → `(t_fixed, 1, 45, 54, 45)` .
- Net effect: whatever the dataset/TR/native resolution, every scan leaves with the SAME shape so a mixed batch stacks.

Where & why it's used:

- **Called by** `MixedFMRIDataset._load` .
- **Why:** the orchestrator bundling the three finishing steps in order (channel fix, spatial resize, temporal resample + z-score), producing the single uniform shape the ViT and collate require.

`_temporal_resample` — polyphase resample to 270 @ 0.72 s

📁 `dinov2/data/fmri_data.py` · lines **238–267** · 🟢 new file

```
def _temporal_resample(clip, n_out):
    n_in = clip.shape[0]
    if n_in == n_out:
        return clip
    g = math.gcd(n_out, n_in)
    out = resample_poly(clip.contiguous().numpy(), n_out // g, n_in // g, axis=0)
    out = torch.from_numpy(np.ascontiguousarray(out)).float()
    if out.shape[0] > n_out:                        # guard off-by-one from ceil
        out = out[:n_out]
    elif out.shape[0] < n_out:
        pad = out[-1:].expand(n_out - out.shape[0], *out.shape[1:])
        out = torch.cat([out, pad], dim=0)
    return out.contiguous()
```

What it does:

- `n_in = clip.shape[0]` : native frame count of the window (≈194.4 s).
- `if n_in == n_out: return clip` : fast no-op (HCP is already 270 @ 0.72 s).

- `g = math.gcd(n_out, n_in)` : reduces the rational resampling factor to lowest terms (`n_out/g` up, `n_in/g` down) — cheaper polyphase filter.
- `resample_poly(..., n_out//g, n_in//g, axis=0)` : SciPy polyphase (anti-aliased FIR) resampling along time to bring `n_in` → `n_out` (e.g. ADNI 65→270). Chosen over linear interpolation because BOLD is band-limited and polyphase avoids the aliasing naive interpolation introduces (per Ariel).
- `torch.from_numpy(np.ascontiguousarray(out)).float()` : back to a contiguous float tensor.
- `if out.shape[0] > n_out: out = out[:n_out]` : trim the +1 frame `resample_poly` can return from rounding.
- `elif out.shape[0] < n_out:` : pad by repeating the last frame (`out[-1:].expand(...)` + `cat`) to land on exactly `n_out` .
- `return out.contiguous()` .

Where & why it's used:

- **Called by** `_finalize` .
- **Why:** the actual TR harmonization — puts every cohort onto the common 0.72 s rate at exactly 270 frames, so mixed windows are temporally comparable/stackable. The trim/pad guard makes the length deterministic despite `resample_poly` rounding.

Phase 4 — Normalization

`_zscore_per_frame` — per-frame spatial z-score

📁 `dinov2/data/fmri_data.py` · lines **182–193** · 🟢 new file

```
def _zscore_per_frame(scan):
    mean = scan.mean(dim=(1, 2, 3, 4), keepdim=True)
    std = scan.std(dim=(1, 2, 3, 4), keepdim=True)
    return torch.where(std > 1e-6, (scan - mean) / std.clamp_min(1e-6), torch.zeros_like(scan))
```

What it does:

- Input `(T, 1, X, Y, Z)` ; standardizes EACH timepoint volume independently across its voxels.
- `mean` : per-frame mean over channel + all spatial dims (1–4), `keepdim=True` to broadcast; frame axis 0 preserved so each of T frames gets its own mean.
- `std` : per-frame std over the same voxels.
- `torch.where(std > 1e-6, (scan - mean)/std.clamp_min(1e-6), zeros)` : where a frame has real variance, z-score it (`clamp_min` guards even the true branch); where a frame is (near-)constant, output zeros instead of NaN/Inf.

Where & why it's used:

- **Called by** `_finalize` , as the OUTERMOST call (run last, after resize + resample).
- **Why:** removes per-frame intensity offset/scale differences (scanner gain, dataset value ranges) so the ViT sees comparable inputs across cohorts. Running it after resampling means stats are computed on the final frames the model consumes. The constant-frame guard keeps degenerate volumes from producing NaNs.

Phase 5 — Augmentation (masking-only)

`FullVolumeViews3D` — the fMRI "views" transform (`__init__` + `__call__`)

📁 `dinov2/data/fmri_data.py` · lines **403–431** · 🟢 new file

```python
class FullVolumeViews3D:
    def __init__(self, local_crops_number, **_ignored):
        self.global_crops_number = GLOBAL_CROPS_NUMBER          # constant (=2), DINO invariant
        self.local_crops_number = int(local_crops_number)
        logger.info(f"fMRI full-volume views: global={self.global_crops_number} "
                    f"local={self.local_crops_number} (masking applied later in collate)")

    def __call__(self, scan):
        return {"global_crops":         [scan] * self.global_crops_number,
                "global_crops_teacher": [scan] * self.global_crops_number,
                "local_crops":          [scan] * self.local_crops_number,
                "offsets":              ()}
```

What it does:

- Drop-in for `DataAugmentationDINO`: same view-dict contract (`global_crops`, `global_crops_teacher`, `local_crops`, `offsets`) so the rest of DINOv2 is untouched.
- `__init__(local_crops_number, **_ignored)`: takes only `local_crops_number` (must equal `cfg.crops.local_crops_number`, the value the DINO loss reads in `ssl_meta_arch` — one source of truth); `int(...)`.
- `self.global_crops_number = GLOBAL_CROPS_NUMBER` — constant (=2), hard-wiring the DINO 2-global-crop invariant.
- `**_ignored` absorbs `DataAugmentationDINO`'s scale/size args so a caller passing them won't crash; dead here because masking-only never crops.
- Logs (once) the counts + the reminder that masking is applied later in the collate.
- `__call__(scan)`: given one preprocessed `(T_fixed, 1, 45, 54, 45)` window, returns the view dict where EVERY view is the SAME object `scan` repeated — 2 global, 2 teacher, N local references; `offsets = ()`.
- Performs NO cropping and NO masking — only builds the multi-view structure; all views alias the identical tensor (not copies).

Where & why it's used:

- **Constructed in** `train.py:334` inside the `elif getattr(cfg.train, "fmri_augmentation", False):` branch; the dict is consumed by `collate_data_and_cast` (`collate.py`).
- **Why:** the fMRI substitute for DINOv2 spatial-crop augmentation. Spatial crops were dropped on purpose (a brain is a fixed anatomical structure, not a scene to crop; meeting §2), so the "augmentation" reduces to structurally-correct view replication, deferring the only real corruption (per-token masking) to the collate.
- **NOTE (** `tasks/v3/FINDINGS.md` **):** because all views are the SAME full volume and no per-view augmentation is applied, DINO/iBOT see NO augmentation gap → the SSL loss does not descend. Verified NOT a code bug (loss computation identical to upstream); the fix is a real student/teacher view gap.

`RandomTokenMaskingGenerator` — MAE-style per-token random masking

📁 `dinov2/data/masking.py` · lines **24–54** · 🟠 modified official file (NEW class; upstream `MaskingGenerator` unchanged)

```python
class RandomTokenMaskingGenerator:
    def __init__(self, input_size, **_ignored):
        if not isinstance(input_size, tuple):
            input_size = (input_size,) * 2
        self.height, self.width = input_size
        self.num_patches = self.height * self.width

    def __repr__(self):   return f"RandomTokenMaskingGenerator({self.height}, {self.width})"
    def get_shape(self):  return self.height, self.width

    def __call__(self, num_masking_patches=0):
        mask = np.zeros(self.num_patches, dtype=bool)
        num = min(int(num_masking_patches), self.num_patches)
        if num > 0:
            idx = np.random.choice(self.num_patches, size=num, replace=False)
            mask[idx] = True
        return mask.reshape(self.height, self.width)
```

What it does:

- A new class added to `masking.py` (the upstream `MaskingGenerator` is left untouched); a drop-in exposing the same `__call__(num) -> (H, W) bool`, `get_shape()`, `__repr__` API.
- `__init__(input_size, **_ignored)`: `input_size` tuple `(height, width)` (a scalar is broadcast to `(n, n)`); stores `height`, `width`, `num_patches = h·w`; `**_ignored` swallows extra kwargs (e.g. `max_num_patches`) for true drop-in compatibility.
- `__call__`: builds a flat all-`False` mask of length `num_patches`; clamps `num = min(request, num_patches)`; if `num > 0` draws `num` DISTINCT indices uniformly (`np.random.choice(..., replace=False)`), sets them `True`; reshapes to `(H, W)`.
- Key difference from `MaskingGenerator`: masks tokens INDEPENDENTLY at random (MAE-style) over the FLATTENED `(T_eff, N_spatial)` grid — no spatial/temporal contiguity, unlike BeiT blocks.

Where & why it's used:

- **Instantiated in** `train.py:301` when `cfg.train.fmri_masking_only`; bound into `collate_data_and_cast` via `partial` and INVOKED PER BATCH in `collate.py` (unchanged upstream) — `mask_generator(int(N·uniform(prob_min, prob_max)))` for masked samples, `mask_generator(0)` otherwise.
- **Why:** the fMRI flattened token order does NOT respect 3D anatomical neighborhoods, so BeiT contiguous-block masking would be spatially incoherent; per-token random masking sidesteps that (meeting §2) and is the ONLY real corruption in the masking-only pipeline.

---

fMRI token-grid & mask-generator wiring in `do_train`

📁 `dinov2/train/train.py` · lines **286–315** (mask-gen), **331–335** (transform), **345–352** (collate) · 🟠 modified official file

```
    if getattr(cfg.train, "fmri_augmentation", False):
        gx, gy, gz = (s // cfg.student.patch_size for s in cfg.student.fmri_img_size)
        n_spatial = gx * gy * gz
        t_eff = cfg.student.fmri_temporal_size // cfg.student.fmri_temporal_kernel
        n_tokens = t_eff * n_spatial
        if getattr(cfg.train, "fmri_masking_only", False):
            mask_generator = RandomTokenMaskingGenerator(input_size=(t_eff, n_spatial))
        else:
            mask_generator = MaskingGenerator(input_size=(t_eff, n_spatial), max_num_patches=int(0.5 * n_tokens))
    else:
        ...  # unchanged official 2D (img/p, img/p) computation
...
    elif getattr(cfg.train, "fmri_augmentation", False):
        data_transform = FullVolumeViews3D(cfg.crops.local_crops_number)
...
    collate_fn = partial(collate_data_and_cast, mask_ratio_tuple=cfg.ibot.mask_ratio_min_max,
        mask_probability=cfg.ibot.mask_sample_probability, n_tokens=n_tokens,
        mask_generator=mask_generator, dtype=inputs_dtype)
```

What it does:

- **Token grid** (gated on `fmri_augmentation`): `gx,gy,gz = (s // patch_size for s in fmri_img_size)`, `n_spatial = gx·gy·gz`; `t_eff = fmri_temporal_size // fmri_temporal_kernel`; `n_tokens = t_eff·n_spatial` — replaces the official scalar `(img_size // patch_size)²` which doesn't apply to a 6D volume.
- **Mask-generator selection:** if `fmri_masking_only`, picks `RandomTokenMaskingGenerator((t_eff, n_spatial))`; else the official BeiT `MaskingGenerator` over the same grid.
- **Unchanged else-branch:** when `fmri_augmentation` is off, restores upstream exactly.
- **Transform selection:** a third branch (symmetric to `cell_augmentation`) → `FullVolumeViews3D(cfg.crops.local_crops_number)`.
- **Collate binding:** `n_tokens` + the selected `mask_generator` (+ iBOT ratios) are frozen into `collate_data_and_cast` via `partial`.
- All fMRI additions use `getattr(cfg.train, ..., False)` so configs without the flags default to upstream.

Where & why it's used:

- **In** `do_train`; the `partial` `collate_fn` feeds the data loader, so every batch `collate_data_and_cast` (UNCHANGED upstream) invokes the bound `mask_generator` with `N = n_tokens`.
- **Why:** the single point adapting DINOv2's iBOT masking from the 2D `(img/p, img/p)` grid to the fMRI 6D `(T_eff, N_spatial)` grid and swapping in per-token masking, keeping the collate untouched.

Phase 6 — Embedding: the 3D+1D patchify

`Conv3Plus1d` — factorised 3D-spatial + 1D-temporal conv

📁 `dinov2/layers/patch_embed_3d_plus_1d.py` · lines **30–66** · 🟢 new file

```
class Conv3Plus1d(nn.Module):
    def __init__(self, in_c, out_c, K_s=3, S_s=1, P_s=1, K_t=3, S_t=1, P_t=1):
        super().__init__()
        self.spatial  = nn.Conv3d(in_c, out_c, kernel_size=K_s, stride=S_s, padding=P_s)
        self.temporal = nn.Conv1d(out_c, out_c, kernel_size=K_t, stride=S_t, padding=P_t)

    def forward(self, x):                                        # x: (B, C, T, X, Y, Z)
        B, _, T, _, _, _ = x.shape
        x = rearrange(x, 'b c t x y z -> (b t) c x y z')         # fold T into batch
        x = self.spatial(x)                                      # (B*T, C', X', Y', Z')
        _, _, X2, Y2, Z2 = x.shape
        x = rearrange(x, '(b t) c x y z -> (b x y z) c t', b=B, t=T)  # fold space into batch
        x = self.temporal(x)                                     # (B*X'Y'Z', C', T')
        x = rearrange(x, '(b x y z) c t -> b c t x y z', b=B, x=X2, y=Y2, z=Z2)
        return x
```

What it does:

- A separable 4D conv over `(T, X, Y, Z)` as `Conv3d` (spatial) then `Conv1d` (temporal) — avoids the missing `Conv4d` and cuts params.
- `__init__`: `spatial` = `Conv3d(in_c, out_c, ...)` with independent spatial kernel/stride/padding; `temporal` = `Conv1d(out_c, out_c, ...)` with independent temporal ones (channel change happens only in the spatial conv).
- **Spatial pass:** `rearrange('b c t x y z -> (b t) c x y z')` folds time into batch → each frame convolved independently → `(B*T, C', X', Y', Z')`; re-reads new spatial dims.
- **Temporal pass:** `rearrange('(b t) c x y z -> (b x y z) c t')` folds batch+space into batch → each voxel's length-`T` series convolved independently → `(B*X'Y'Z', C', T')`.
- **Refold:** back to 6D for the next block.

Where & why it's used:

- **Instantiated in** `_ResBlock3Plus1d` (`conv1` / `conv2`) and directly as `conv_in` / `down_0` / `down_1` in `PatchEmbed3DPlus1D`.
- **Why:** the basic building block at every level of the fMRI encoder — the analogue of MovieGen's `Conv2Plus1d`, spatio-temporal mixing with standard PyTorch primitives.

`_ResBlock3Plus1d` + `PatchEmbed3DPlus1D.__init__` + `_encoder_forward` — the hierarchical stack

📁 `dinov2/layers/patch_embed_3d_plus_1d.py` · lines **69–83, 118–194, 212–251** · 🟢 new file

```
class _ResBlock3Plus1d(nn.Module):
    def __init__(self, ch):
        super().__init__()
        self.norm1 = nn.GroupNorm(min(8, ch), ch); self.conv1 = Conv3Plus1d(ch, ch)
        self.norm2 = nn.GroupNorm(min(8, ch), ch); self.conv2 = Conv3Plus1d(ch, ch)
    def forward(self, x):
        h = self.conv1(F.silu(self.norm1(x)))
        h = self.conv2(F.silu(self.norm2(h)))
        return x + h                                          # residual

# PatchEmbed3DPlus1D.__init__ (channels 1 -> 32 -> 64 -> embed_dim):
        self.conv_in = Conv3Plus1d(in_chans, 32, K_s=3, S_s=1, P_s=1, K_t=3, S_t=1, P_t=1)
        self.block_0 = _ResBlock3Plus1d(32)
        self.down_0  = Conv3Plus1d(32, 64, K_s=3, S_s=1, P_s=1, K_t=3, S_t=1, P_t=1)
        self.block_1 = _ResBlock3Plus1d(64)
        down_1_kt = temporal_kernel // 2                      # derived from the config
        self.down_1  = Conv3Plus1d(64, embed_dim, K_s=3, S_s=1, P_s=1, K_t=3, S_t=1, P_t=1)
        self.spool_k = 3; self.pool0_k, self.pool1_k = 2, down_1_kt
        self.block_2 = _ResBlock3Plus1d(embed_dim)
        gx, gy, gz = (s // patch_size for s in self.img_size)
        self.num_spatial_patches  = gx * gy * gz
        self.num_temporal_patches = temporal_size // temporal_kernel
        self.num_patches = self.num_temporal_patches * self.num_spatial_patches
        self.pos = PositionEmbedding3D(self.num_temporal_patches, self.num_spatial_patches, embed_dim)

# _encoder_forward + forward:
    def _encoder_forward(self, x):
        x = self.conv_in(x); x = self._spool(x, self.spool_k)   # spatial /3
        x = self.block_0(x); x = self.down_0(x)
        x = self._spool(x, self.spool_k); x = self._tpool(x, self.pool0_k)  # spatial /3, temporal /2
        x = self.block_1(x); x = self.down_1(x)
        x = self._tpool(x, self.pool1_k)                      # temporal /down_1_kt
        x = self.block_2(x); return x
    def forward(self, x):                                     # (B, T, C, X, Y, Z)
        if x.ndim != 6: raise ValueError(...)
        x = rearrange(x, 'b t c x y z -> b c t x y z')
        if self.training:
            x = torch.utils.checkpoint.checkpoint(self._encoder_forward, x, use_reentrant=False)
        else:
            x = self._encoder_forward(x)
        return rearrange(x, 'b c t x y z -> b (t x y z) c')     # (B, 4050, embed_dim)
```

What it does:

- `_ResBlock3Plus1d`: pre-norm residual block, two `Conv3Plus1d` each preceded by `GroupNorm(min(8,ch), ch)` + SiLU; `forward` = `x + conv2(silu(norm2(conv1(silu(norm1(x))))))`. `min(8,ch)` keeps groups valid for small channel counts.
- `__init__` **contract:** the dinov2 `PatchEmbed` signature (`img_size, patch_size, in_chans, embed_dim`) plus fMRI `temporal_size` / `temporal_kernel`.
- **Hierarchical stack (1→32→64→384):** `conv_in` (stride-1), `block_0` @32, `down_0` (→64), `block_1` @64, `down_1` (→embed_dim), `block_2` @embed_dim — all convs stride-1.
- **Downsample factors, derived:** `spool_k=3` (spatial AvgPool ×2 = /9 = patch_size), `pool0_k=2` (temporal /2), `pool1_k = down_1_kt = temporal_kernel//2`. Product `1·2·(kernel//2) = temporal_kernel`, so the architecture is a pure function of `temporal_kernel` and rebuildable from a saved config.
- **Token sizing:** `gx,gy,gz = (s//patch_size)` ; `num_spatial_patches = gx·gy·gz` (5·6·5=150); `num_temporal_patches = temporal_size//temporal_kernel` (27); `num_patches = 27·150 = 4050`. `num_patches` is exposed for the parent ViT `__init__` (its fMRI-unused flat pos_embed).
- **Positional module:** builds one `PositionEmbedding3D` as `self.pos`.
- `_encoder_forward`: realizes spatial 45→15→5 and temporal 270→135→27 via `conv_in → spool → block_0 → down_0 → spool → tpool → block_1 → down_1 → tpool → block_2`.
- `forward`: validates 6D (else `ValueError`); permutes `(B,T,C,X,Y,Z)→(B,C,T,X,Y,Z)` ; runs `_encoder_forward` under `torch.utils.checkpoint(..., use_reentrant=False)` when training (recompute-in-backward to save memory) else directly; flattens to tokens `'b c t x y z -> b (t x y z) c'` (t outer, space inner — matching `combined_patch_pos`).

Where & why it's used:

- **Turned into a `partial` in `models/__init__.py`** and passed as the ViT `embed_layer`; the ViT calls `self.patch_embed(x)` in the 6D branch.
- **Why:** the entire fMRI patchifier — a raw 4D volume-time tensor → the fixed 4050-token sequence the transformer consumes, with reductions/token-count derived purely from config.

`_spool` / `_tpool` — AvgPool downsampling on stride-1 conv outputs

📁 `dinov2/layers/patch_embed_3d_plus_1d.py` · lines **196–210** · 🟢 new file

```
    @staticmethod
    def _tpool(x, k):                                       # (B,C,T,X,Y,Z) -> temporal /k
        B, C, T, X, Y, Z = x.shape
        x = rearrange(x, 'b c t x y z -> (b c x y z) t').unsqueeze(1)   # (N,1,T)
        x = F.avg_pool1d(x, kernel_size=k, stride=k).squeeze(1)         # (N, T//k)
        return rearrange(x, '(b c x y z) t -> b c t x y z', b=B, c=C, x=X, y=Y, z=Z)

    @staticmethod
    def _spool(x, k):                                       # (B,C,T,X,Y,Z) -> spatial /k
        B, C, T, X, Y, Z = x.shape
        x = rearrange(x, 'b c t x y z -> (b c t) x y z').unsqueeze(1)   # (N,1,X,Y,Z)
        x = F.avg_pool3d(x, kernel_size=k, stride=k).squeeze(1)         # (N, X/k, Y/k, Z/k)
        return rearrange(x, '(b c t) x y z -> b c t x y z', b=B, c=C, t=T)
```

What it does:

- Both `@staticmethod`, take a 6D tensor + factor `k`, do non-overlapping average pooling (kernel= `k`, stride= `k`).
- **`_tpool`:** folds every non-temporal axis into batch, `.unsqueeze(1)` → `(N,1,T)`, `F.avg_pool1d(k,k)`, `.squeeze(1)`, refold → `(B,C,T//k,X,Y,Z)`.
- **`_spool`:** folds `(B,C,T)` into batch, `.unsqueeze(1)` → `(N,1,X,Y,Z)`, `F.avg_pool3d(k,k)`, `.squeeze(1)`, refold → `(B,C,T,X/k,Y/k,Z/k)`.
- The fold-into-batch trick lets 1D/3D pooling act on exactly the intended axes without a 4D pooling primitive; both preserve the 6D layout.

Where & why it's used:

- **Called only from** `_encoder_forward` (`_spool` ×2, `_tpool` ×2).
- **Why:** they realize all spatial (45→15→5) and temporal (270→135→27) downsampling. Every conv stays stride-1 and reduction is deferred to these pools, so the final token grid is a pure function of img_size/patch_size and temporal_size/temporal_kernel.

`PositionEmbedding3D` — factorised learned positional embedding, O(T+N) params

📁 `dinov2/layers/patch_embed_3d_plus_1d.py` · lines **86–115** · 🟢 new file

```python
class PositionEmbedding3D(nn.Module):
    def __init__(self, num_temporal_patches, num_spatial_patches, embed_dim):
        super().__init__()
        self.num_temporal_patches = num_temporal_patches
        self.num_spatial_patches  = num_spatial_patches
        self.pos_temporal = nn.Parameter(torch.zeros(1, num_temporal_patches, embed_dim))
        self.pos_spatial  = nn.Parameter(torch.zeros(1, num_spatial_patches, embed_dim))
        self.pos_cls      = nn.Parameter(torch.zeros(1, 1, embed_dim))
        trunc_normal_(self.pos_temporal, std=0.02)
        trunc_normal_(self.pos_spatial,  std=0.02)
        trunc_normal_(self.pos_cls,      std=0.02)

    def combined_patch_pos(self):                              # (1, T_eff*N_spatial, D)
        pos_t = repeat(self.pos_temporal, '1 t d -> 1 (t n) d', n=self.num_spatial_patches)
        pos_s = repeat(self.pos_spatial,  '1 n d -> 1 (t n) d', t=self.num_temporal_patches)
        return pos_t + pos_s
```

What it does:

- `__init__` stores the two counts and creates three learned tables: `pos_temporal (1, T_eff, D)`, `pos_spatial (1, N_spatial, D)`, `pos_cls (1, 1, D)` — total O(T_eff + N_spatial) params instead of O(T_eff · N_spatial) for a flat table. All zero-init then `trunc_normal_(std=0.02)` (dinov2 convention).
- `combined_patch_pos` broadcasts `pos_temporal` across space and `pos_spatial` across time, summing into `(1, T_eff·N_spatial, D)`. The `(t n)` grouping fixes token order as time-outer, space-inner — matching the flatten order in `PatchEmbed3DPlus1D.forward`. `pos_cls` is NOT folded in here — the ViT adds it to the CLS token separately.

Where & why it's used:

- **Instantiated as** `self.pos` in `PatchEmbed3DPlus1D.__init__`; `combined_patch_pos()` and `pos_cls` are read by the ViT 6D branch.
- **Why:** the fMRI positional signal. Factorising temporal vs spatial keeps the parameter budget tiny for a 4050-token grid; `pos_cls` gives the class token its own learned position.

`build_model_from_cfg` fMRI branch + `build_model` embed_layer forwarding

📁 `dinov2/models/__init__.py` · lines **25, 43–57, 70–97** · 🟠 modified official file

```python
def build_model(args, only_teacher=False, img_size=224, embed_layer=None):  # FMRI: added embed_layer
    ...
        if embed_layer is not None:                            # FMRI: forward to the ViT
            vit_kwargs["embed_layer"] = embed_layer
        teacher = vits.__dict__[args.arch](**vit_kwargs)
...
def build_model_from_cfg(cfg, only_teacher=False):
    embed_layer = None
    img_size = cfg.crops.global_crops_size
    if getattr(cfg.student, "fmri_mode", False):
        from functools import partial
        from dinov2.layers.patch_embed_3d_plus_1d import PatchEmbed3DPlus1D
        embed_layer = partial(PatchEmbed3DPlus1D,
            temporal_size=cfg.student.fmri_temporal_size,
            temporal_kernel=cfg.student.fmri_temporal_kernel)
        img_size = tuple(cfg.student.fmri_img_size)
    return build_model(cfg.student, only_teacher=only_teacher, img_size=img_size, embed_layer=embed_layer)
```

What it does:

- **`build_model`:** adds an `embed_layer=None` param; if not `None`, injects it into `vit_kwargs["embed_layer"]` before both teacher and student are built via `vits.__dict__[args.arch](**vit_kwargs)`. `None` → identical to upstream (ViT picks its default 2D `PatchEmbed`).
- **`build_model_from_cfg`:** defaults `embed_layer=None`, `img_size=cfg.crops.global_crops_size` (upstream). fMRI branch gated by `getattr(cfg.student, "fmri_mode", False)`: lazily imports `PatchEmbed3DPlus1D` + `partial`, binds `temporal_size` / `temporal_kernel` (leaving `img_size` for the ViT's `**vit_kwargs`), and overrides `img_size = tuple(cfg.student.fmri_img_size)` (the 3D `(X,Y,Z)` instead of a scalar). Returns `build_model(..., img_size, embed_layer)`.

Where & why it's used:

- **Called by** `SSLMetaArch`; the `partial` becomes the ViT `embed_layer`, consumed inside `DinoVisionTransformer.__init__` as `self.patch_embed = embed_layer(img_size, patch_size, in_chans, embed_dim)`.
- **Why:** the single config-driven switch turning the stock 2D DINOv2 into the 4D fMRI model — all plumbing stays in `models/__init__.py` + the config, no changes to `SSLMetaArch` or `do_train`.

`prepare_tokens_with_masks` 6D branch — masking + factorised pos + CLS/register tokens

📁 `dinov2/models/vision_transformer.py` · lines **243–263** (branch within the method) · 🟠 modified official file

```
        if x.ndim == 6:
            x = self.patch_embed(x)                                    # (B, 4050, D)
            if masks is not None:                                      # mask BEFORE pos (as official)
                x = torch.where(masks.unsqueeze(-1), self.mask_token.to(x.dtype).unsqueeze(0), x)
            x = x + self.patch_embed.pos.combined_patch_pos()          # factorised pos
            cls = self.cls_token.expand(x.shape[0], -1, -1) + self.patch_embed.pos.pos_cls
            x = torch.cat((cls, x), dim=1)                             # prepend CLS (+ its pos)
            if self.register_tokens is not None:                       # insert registers (no pos)
                x = torch.cat((x[:, :1], self.register_tokens.expand(x.shape[0], -1, -1), x[:, 1:]), dim=1)
            return x
```

What it does:

- Early-return branch guarded by `if x.ndim == 6` (fMRI 6D input); the untouched upstream 4D path continues below.
- **Patchify:** `self.patch_embed(x)` → `(B, 4050, D)`.
- **Masking (before pos):** if `masks is not None`, replaces masked positions with the learned `mask_token` via `torch.where(...)` — before adding positions, matching the official 4D ordering (pure content substitution).
- **Factorised patch pos:** adds `self.patch_embed.pos.combined_patch_pos()`, broadcast over batch — uses the factorised table, NOT the flat `self.pos_embed` (still allocated by the parent `__init__` but dead weight in fMRI mode).
- **CLS token:** `cls = self.cls_token.expand(B,-1,-1) + self.patch_embed.pos.pos_cls` (CLS gets its own position), prepended with `torch.cat`.
- **Register tokens:** if present, spliced right after the CLS token (registers get no positional embedding — official convention).
- Returns `[CLS, (registers), patches]` and short-circuits the rest.

Where & why it's used:

- **Invoked by** `forward_features_list`, `forward_features`, and the intermediate-layer getters; the 6D dispatch fires whenever `PatchEmbed3DPlus1D` feeds 6D volumes.
- **Why:** the join point between the fMRI patch embed and the transformer trunk — assembling masked patch tokens + factorised pos + CLS/register tokens into the exact sequence the DINOv2 blocks expect, while leaving the 2D image path byte-for-byte upstream.

Phase 7 — DINO / SSL training

`apply_freeze_policy` — partial-freeze ablation of the student backbone

📁 `dinov2/train/train.py` · lines **156–205** (function) + **254** (call) · 🟠 modified official file

```python
def apply_freeze_policy(model, freeze_mode):
    if freeze_mode in (None, "none", ""):
        return                                              # no freeze = official behavior
    backbone = model.student.backbone
    n_frozen = n_train = 0
    for name, p in backbone.named_parameters():
        logical = name.replace("_fsdp_wrapped_module.", "")    # strip FSDP wrap for prefix match
        if freeze_mode == "fmri_only":
            trainable = logical.startswith("patch_embed.")
        elif freeze_mode == "fmri_plus_last_3":
            trainable = (logical.startswith("patch_embed.")
                or logical.startswith("blocks.9.") or logical.startswith("blocks.10.")
                or logical.startswith("blocks.11.") or logical.startswith("norm."))
        else:
            raise ValueError(f"Unknown freeze_pretrained mode: {freeze_mode!r}")
        p.requires_grad_(trainable)
        n_train += p.numel() if trainable else 0
        n_frozen += 0 if trainable else p.numel()
    logger.info(f"FMRI freeze_pretrained={freeze_mode!r}: trainable={n_train:,} frozen={n_frozen:,}")

# call in do_train (L254):
    apply_freeze_policy(model, getattr(cfg.optim, "freeze_pretrained", None))
```

What it does:

- A 3-mode freeze ablation over the student BACKBONE only; the DINO/iBOT heads are random-init and always stay trainable (the SSL loss would be meaningless if frozen).
- `None` / `"none"` / `""` → early-return, identical to upstream.
- Iterates `model.student.backbone.named_parameters()`, stripping every `_fsdp_wrapped_module.` segment to recover the logical module path (works whether or not FSDP-wrapped).
- `"fmri_only"` → trainable only where the logical path starts with `patch_embed.` (Conv3Plus1d stack + 3D pos); the rest frozen.
- `"fmri_plus_last_3"` → also keeps `blocks.9/10/11` + final `norm.` trainable; blocks 0–8, cls_token, register_tokens, pos_embed frozen.
- Any other string → `ValueError` (fail-fast on a typo).
- Calls `p.requires_grad_(trainable)` on every param, tallies trainable vs frozen, logs both.
- The call in `do_train` runs BEFORE `build_optimizer`, so the optimizer only receives params still requiring grad; `getattr(..., None)` default = official full-training for configs without the flag.

Where & why it's used:

- **Called once** in `do_train` (before the optimizer is built).
- **Why:** the freeze ablation (variants A/B/C) — lets a filtered ImageNet DINOv2 backbone be partially frozen while the fMRI `patch_embed` and heads adapt, isolating how much of the pretrained transformer to fine-tune on fMRI.

`elif getattr(cfg.train, "fmri_augmentation", False)` — fMRI augmentation-selection branch

📁 `dinov2/train/train.py` · lines **331–335** (within the `if/elif/else` at 317–343) · 🟠 modified official file

```python
    if cfg.train.cell_augmentation:
        data_transform = CellAugmentationDINO(...)
    elif getattr(cfg.train, "fmri_augmentation", False):
        data_transform = FullVolumeViews3D(cfg.crops.local_crops_number)
    else:
        data_transform = DataAugmentationDINO(...)
```

What it does:

- Adds a third mutually-exclusive augmentation branch (symmetric to the official `cell_augmentation`), selecting `FullVolumeViews3D` when `cfg.train.fmri_augmentation` is truthy.
- Passes only `cfg.crops.local_crops_number`: masking-only ignores crop scale/size, and the global count is a class constant (`GLOBAL_CROPS_NUMBER = 2`).

- Sits between the `cell_augmentation` branch and the official `DataAugmentationDINO` else, so official behavior is preserved when the flag is absent.
- **The `getattr(..., False)` default:** `cfg.train.fmri_augmentation` is absent from the upstream config schema; a direct attribute access on a pre-fork config would raise `AttributeError`. `getattr(..., False)` returns `False` for any config lacking the key → the branch is skipped and the fork stays backward-compatible with every upstream config.

Where & why it's used:

- **In** `do_train`; the resulting `data_transform` is handed to `make_dataset(..., transform=...)`.
- **Why:** fMRI volumes are 3D+time; the official 2D crop/color augmentations are meaningless — this swaps in the masking-only full-volume view builder while leaving both official paths untouched.

Gradient accumulation — cycle guard + `optimizer_step_and_ema` + `loss_scale` plumbing

📁 `dinov2/train/train.py` · lines **410–440, 213–235** · 🟠 · and `dinov2/train/ssl_meta_arch.py` · lines **146–151, 361–366, 372–397** · 🟠

```
# do_train guard (train.py):
        grad_accum_steps = int(cfg.optim.get("grad_accum_steps", 1))
        if iteration % grad_accum_steps == 0:
            optimizer.zero_grad(set_to_none=True)                # zero at cycle START
        loss_dict = model.forward_backward(data, teacher_temp=teacher_temp, loss_scale=float(grad_accum_steps))
        if (iteration + 1) % grad_accum_steps == 0:
            optimizer_step_and_ema(model, optimizer, fp16_scaler, cfg.optim.clip_grad, mom)  # step at cycle END

# optimizer_step_and_ema (train.py):
def optimizer_step_and_ema(model, optimizer, fp16_scaler, clip_grad, mom):
    if fp16_scaler is not None:
        if clip_grad:
            fp16_scaler.unscale_(optimizer)                      # 1. undo fp16 loss-scaling
            for v in model.student.values(): v.clip_grad_norm_(clip_grad)   # 2. clip
        fp16_scaler.step(optimizer); fp16_scaler.update()        # 3. step STUDENT (skips on inf/nan)
    else:
        if clip_grad:
            for v in model.student.values(): v.clip_grad_norm_(clip_grad)
        optimizer.step()
    model.update_teacher(mom)                                    # 4. teacher = EMA of student

# ssl_meta_arch.py:
    def forward_backward(self, images, teacher_temp, loss_scale: float = 1.0):
        ...
        self.backprop_loss(loss_accumulator / loss_scale)        # divide by N before backward
    # _streams guard in fsdp_synchronize_streams:
            if hasattr(self.teacher.backbone, "_streams"):
                self.student.dino_head._streams = ... = self.teacher.backbone._streams
```

What it does:

- `grad_accum_steps` read per-iteration via `.get("grad_accum_steps", 1)`, cast to int (default 1 = upstream single step).
- **Cycle start:** `zero_grad(set_to_none=True)` only when `iteration % N == 0`, so gradients ACCUMULATE across the N micro-steps instead of clearing every step.
- **Every micro-step:** `forward_backward(loss_scale=N)` → `backprop_loss(loss_accumulator / loss_scale)` divides the loss by N before `.backward()`; summed over N backwards = one full-batch gradient. `loss_dict` stays unscaled so printed losses match a single-step run.
- **Cycle end:** when `(iteration + 1) % N == 0`, `optimizer_step_and_ema` runs once: (fp16 `unscale_` →) `clip_grad_norm_` per student submodule → `optimizer.step()` (the fp16 scaler skips the step on inf/nan overflow) → `update_teacher(mom)` EMA.
- Extracting the step body keeps the accumulation guard a readable two-liner; the logic is unchanged from upstream. `loss_scale=1.0` default makes `forward_backward` a drop-in.
- **`_streams` guard:** the one-shot FSDP stream-sharing assignment is wrapped in `if hasattr(self.teacher.backbone, "_streams")`; PyTorch ≥ ~2.3 removed that attribute, so the guard skips the workaround on newer torch (FSDP syncs streams itself) while preserving it for older versions.

Where & why it's used:

- **In** the `do_train` loop; `optimizer_step_and_ema` (1 call), `forward_backward` (1 call), `fsdp_synchronize_streams` (end of `forward_backward`).
- **Why:** fMRI volumes are large → per-GPU micro-batch is only 2; accumulating over 8 gives an effective batch of 16 without exceeding memory, while `loss_scale` keeps the gradient identical to a true batch-16 step. The `_streams` guard is required because this fork runs on a newer PyTorch than upstream assumed.

`apply_scaling_rules_to_cfg` — grad-accum folded into effective batch for LR scaling

📁 `dinov2/utils/config.py` · lines **30–50** · 🟠 modified official file

```python
def apply_scaling_rules_to_cfg(cfg):
    if cfg.optim.scaling_rule == "sqrt_wrt_1024":
        base_lr = cfg.optim.base_lr
        grad_accum = int(cfg.optim.get("grad_accum_steps", 1))           # FMRI: include grad_accum
        effective_batch = cfg.train.batch_size_per_gpu * distributed.get_global_size() * grad_accum
        cfg.optim.lr = base_lr * math.sqrt(effective_batch / 1024.0)
        logger.info(f"sqrt scaling ... base: {base_lr}, grad_accum: {grad_accum}, "
                    f"effective_batch: {effective_batch}, new lr: {cfg.optim.lr}")
    else:
        raise NotImplementedError
    return cfg
```

What it does:

- Implements the `sqrt_wrt_1024` rule; any other `scaling_rule` → `NotImplementedError` (upstream).
- Reads `grad_accum_steps` via `.get(..., 1)` (default 1).
- Computes `effective_batch = batch_size_per_gpu * global_size * grad_accum` — folding in the accumulation factor (upstream used only `batch_size_per_gpu * global_size`).
- Sets `cfg.optim.lr = base_lr * sqrt(effective_batch / 1024)`; logs base_lr, grad_accum, effective_batch, lr.

Where & why it's used:

- **Called by** `setup(args)` (config.py), the entry invoked by `main` in `train.py`.
- **Why:** with micro-batch 2 × grad_accum 8 the true optimization batch is 16, not 2. Without this the sqrt rule would scale the LR from a batch of 2 (~3× too small). Folding grad_accum in makes the LR reflect the real effective batch (`3e-4 * sqrt(16/1024) ≈ 3.75e-5`), so accumulation is transparent to the schedule.

`fmri_vits.yaml` — fMRI training knobs

📁 `dinov2/configs/train/fmri_vits.yaml` · lines **8–104** · 🔵 config

```yaml
train:
  dataset_path: Mixed                # 5-source corpus, 4627 scans @ T_fixed=270
  batch_size_per_gpu: 2              # micro-batch 2; effective 16 via grad_accum=8
  OFFICIAL_EPOCH_LENGTH: 2300        # drives the LR/wd/momentum/temp schedules
  centering: "sinkhorn_knopp"
  fmri_augmentation: true            # fMRI token grid + FullVolumeViews3D branch
  fmri_masking_only: true            # per-token random masking
  proportional_sampler: true         # per-batch quota HCP4/ABIDE4/OASIS4/ADNI3/AOMIC1
student:
  arch: vit_small
  patch_size: 9                      # spatial Conv3d kernel/stride
  fmri_mode: true
  fmri_img_size: [45, 54, 45]        # patch=9 -> 5*6*5 = 150 spatial tokens
  fmri_temporal_size: 270            # T_fixed window
  fmri_temporal_kernel: 10           # T_eff = 27 temporal tokens
teacher:
  momentum_teacher: 0.992  final_momentum_teacher: 1.0
  warmup_teacher_temp: 0.04  teacher_temp: 0.07  warmup_teacher_temp_epochs: 5
optim:
  freeze_pretrained: "fmri_plus_last_3"  grad_accum_steps: 8  epochs: 10
  weight_decay: 0.04  weight_decay_end: 0.4  base_lr: 3.0e-4  warmup_epochs: 0.3
  scaling_rule: sqrt_wrt_1024  patch_embed_lr_mult: 0.2  clip_grad: 3.0
dino:  { loss_weight: 1.0, head_n_prototypes: 4096, koleo_loss_weight: 0.1 }
ibot: { loss_weight: 1.0, mask_sample_probability: 0.5, mask_ratio_min_max: [0.1, 0.5],
        separate_head: false, head_n_prototypes: 4096 }
```

What it does:

- **fMRI geometry:** `fmri_mode`, `fmri_img_size [45,54,45]`, `patch_size 9` (→150 spatial), `fmri_temporal_size 270` + `fmri_temporal_kernel 10` (→27 temporal), total 4050 tokens; consumed by `build_model_from_cfg` and the `do_train` token-grid path.
- **Augmentation/masking:** `fmri_augmentation` selects `FullVolumeViews3D`; `fmri_masking_only` selects per-token `RandomTokenMaskingGenerator`.
- **Sampler:** `proportional_sampler` (default quota HCP4/ABIDE4/OASIS4/ADNI3/AOMIC1 = 16).
- **Freeze/grad-accum:** `freeze_pretrained: "fmri_plus_last_3"` drives `apply_freeze_policy`; `grad_accum_steps 8` × `batch_size_per_gpu 2` = effective 16.
- **LR/schedule:** `scaling_rule sqrt_wrt_1024`, `base_lr 3e-4` (→ ≈3.75e-5), `epochs 10`, `warmup_epochs 0.3` (fractional — motivates the `int()` cast, §7.6), `patch_embed_lr_mult 0.2`, `weight_decay 0.04→0.4`. `OFFICIAL_EPOCH_LENGTH 2300` drives scheduler lengths.
- **Teacher-temp:** `warmup_teacher_temp 0.04 → teacher_temp 0.07` over 5 epochs; `momentum 0.992→1.0`; `centering sinkhorn_knopp`.
- **DINO/iBOT heads:** both 4096 prototypes; DINO `koleo_loss_weight 0.1`; iBOT `separate_head: false` (shares the DINO head), `mask_sample_probability 0.5`, `mask_ratio_min_max [0.1, 0.5]`.

Where & why it's used:

- **Loaded by** `get_cfg_from_args` (config.py), merged onto the default config, consumed throughout `do_train`, `SSLMetaArch.__init__`, `apply_scaling_rules_to_cfg`. Passed via `--config-file`.
- **Why:** the single 🔵 file — it only OVERRIDES values on the official schema (no new schema), so upstream code sees a structurally identical config and only the fMRI knobs differ. It ties together every 🟠 change above.

`build_schedulers` — `int()` casts on the four scheduler `*_iters`

📁 `dinov2/train/train.py` · lines **92–124** · 🟠 modified official file *(setup helper, runs before the loop)*

```
def build_schedulers(cfg):
    OFFICIAL_EPOCH_LENGTH = cfg.train.OFFICIAL_EPOCH_LENGTH
    lr = dict(..., total_iters=int(cfg.optim["epochs"] * OFFICIAL_EPOCH_LENGTH),
                   warmup_iters=int(cfg.optim["warmup_epochs"] * OFFICIAL_EPOCH_LENGTH), ...)
    wd = dict(..., total_iters=int(cfg.optim["epochs"] * OFFICIAL_EPOCH_LENGTH))
    momentum = dict(..., total_iters=int(cfg.optim["epochs"] * OFFICIAL_EPOCH_LENGTH))
    teacher_temp = dict(..., total_iters=int(cfg.teacher["warmup_teacher_temp_epochs"] * OFFICIAL_EPOCH_LENGTH),
                             warmup_iters=int(cfg.teacher["warmup_teacher_temp_epochs"] * OFFICIAL_EPOCH_LENGTH), ...)
```

What it does:

- Builds the five cosine schedules (lr, wd, momentum, teacher_temp, last_layer_lr) as upstream, but wraps the four derived iteration counts in `int()`: `lr.total_iters`, `lr.warmup_iters` (the load-bearing one), `wd.total_iters`, `momentum.total_iters`, and `teacher_temp.total_iters` / `.warmup_iters`. All other fields unchanged; the `lr` dict is reused for both `lr_schedule` and `last_layer_lr_schedule`.

Where & why it's used:

- **Called by** `do_train` (unpacks the five schedules); the schedules feed `CosineScheduler`, which internally uses `np.linspace(..., num=warmup_iters)`.
- **Why:** the fMRI config uses fractional epochs (`warmup_epochs: 0.3`); `0.3 * 2300 = 690.0` is a float and `np.linspace` requires an integer `num` (a float raises `TypeError`). The `int()` casts truncate to a valid iteration count. With integer epochs they are no-ops, so upstream behavior is preserved.